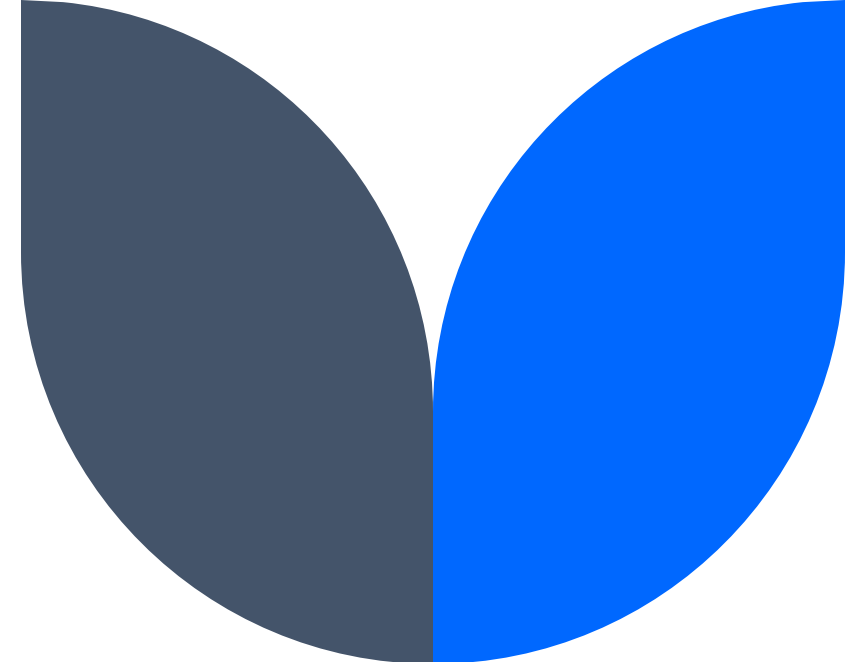
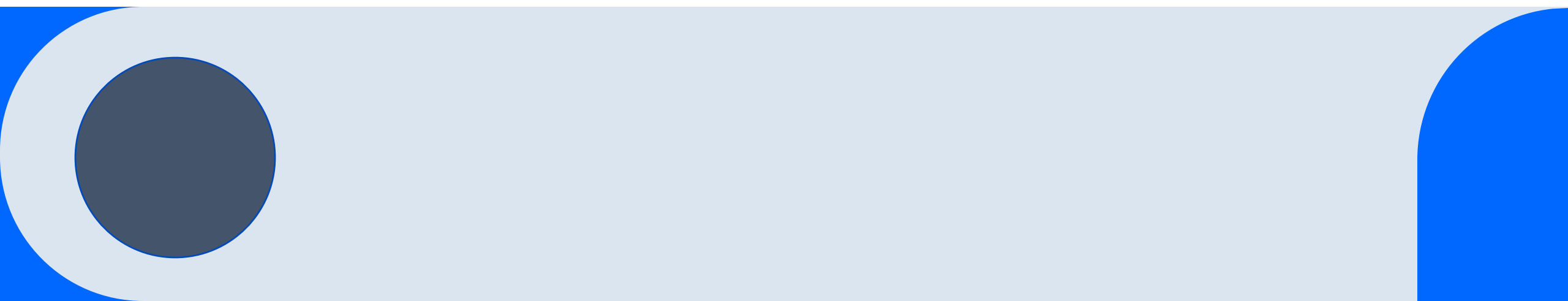




Covid-19 Model Prediction

By: Poorvi Raut



Agenda

1. Project Definition
2. DataSet Overview
3. Data Cleaning and Preprocessing
4. Machine Learning Models
5. Learning and Future Enhancements

1. Project Definiton

- The project aims to determine whether a **COVID-19 patient will survive or not** based on key health indicators. Predicting survival requires analyzing multiple factors, including **age, oxygen levels, severity, and underlying conditions**.
- COVID-19 poses a serious threat to **public health**, and accurate predictions can help hospitals allocate resources effectively. The goal is to develop a system that can **precisely classify patients** based on survival likelihood, aiding in better medical decision-making.



2.DataSet Overview

Reference: [Titanic DataSet Kaggle](#)

This dataset is structured similarly to the **Titanic survival prediction dataset**, where the goal is to classify whether a COVID-19 patient **survives (1) or does not survive (0)** based on medical and demographic features.

- **Each row** represents a **single patient case**
- **Each column** represents a **key attribute** such as **Age, Gender, Oxygen Level, Severity, and Treatment Factors**



DataSet Overview Continued:

`final_train.csv`

- **Contains** training data for model development
- **Rows:891**
- **Columns: 12** features including **Survived (Target Variable)**

`final_test.csv`

- **Used for evaluating model performance**
- **Rows: 418**
- **Columns: Same as final_train but without Survived so 11**

`final_gender_submission.csv`

- **Contains sample predictions for final_test patients**
- **Rows: 418 (Same as final_test)**
- **Columns: 2 Patient ID, Predicted Survival (0/1)**

DataSet Features Overview

Variable	Description	final_train.csv	final_test.csv	final_gender_submission.csv
patient_id	Unique identifier for each patient	✓	✓	✓
age	Age of the patient	✓	✓	✗
gender	Sex of the patient (Male/Female)	✓	✓	✗
hospital_location	Hospital where the patient was admitted	✓	✓	✗
oxygen_level	Oxygen saturation level (%)	✓	✓	✗
severity_level	Severity of COVID-19 (Mild/Moderate/Severe)	✓	✓	✗
pre_existing_cond	Any pre-existing medical conditions (Yes/No)	✓	✓	✗
ventilator_needed	Whether the patient required a ventilator (Yes/No)	✓	✓	✗
treatment_cost	Total cost of medical treatment (in USD)	✓	✓	✗
hospital_stay_days	Number of days admitted to the hospital	✓	✓	✗
city	City where the patient is receiving treatment	✓	✓	✗
state	State where the hospital is located	✓	✓	✗
zip_code	ZIP code of the hospital	✓	✓	✗
is_survived	Target variable: Patient survival status (1/0)	✓	✗	✓ (Predicted)

3. Data Cleaning and Preprocessing

Data cleaning and preprocessing is a crucial step in Data Science for model prediction. It involves identifying and correcting errors in the dataset by handling missing values, removing duplicates, and fixing inconsistencies. It ensures the data is accurate, consistent, and ready for analysis or machine learning.

Steps Involved include:

1. Handling missing values and Feature encoding
2. Feature Engineering
3. Exploratory Data Analysis



3.1 Handling missing values and Feature Encoding

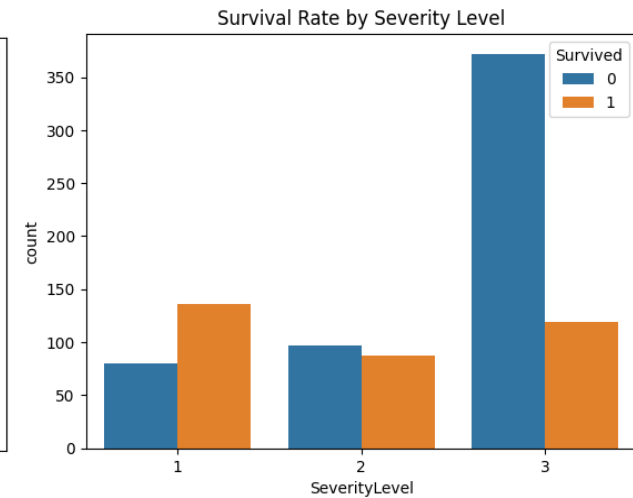
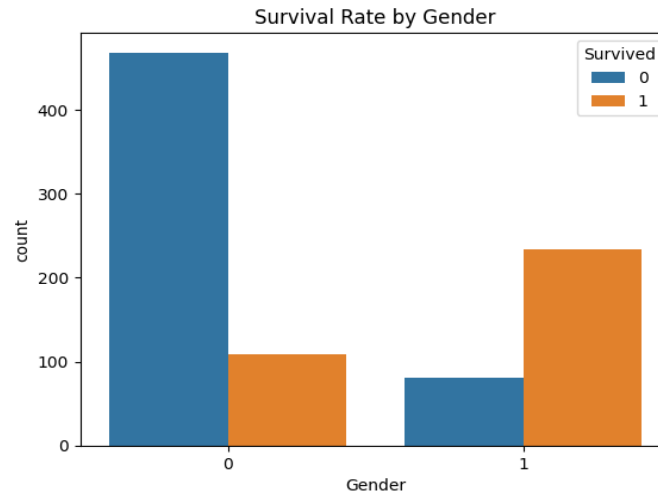
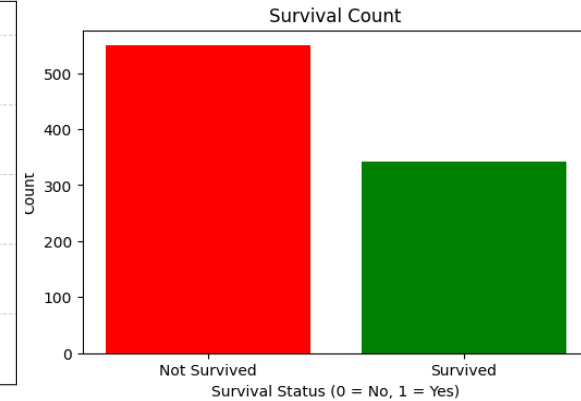
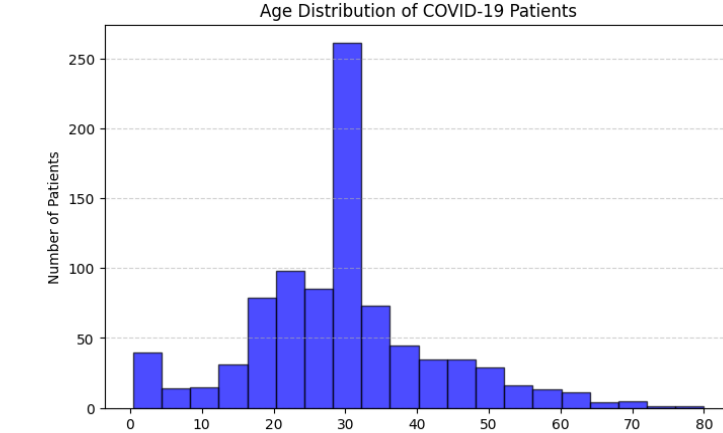
Under **Data Cleaning & Preprocessing**, the **Handling Missing Values** step ensures data completeness by addressing gaps in the dataset.

For our given **final_train.csv** and **final_test.csv** files

1. checked for any missing values as first step. `final_train.isnull().values.any()`
2. We filled all numerical numerical with **mean** and all categorical columns with **mode**
3. Converted Categorical variables to numeric data like **Gender** (Male/Female → 0/1) and **HospitalLocation** {'C': 0, 'Q': 1, 'S': 2}
4. Finally Dropped few irrelevant columns from **final_train.csv** and **final_test.csv** files like **'PatientID', 'FullName', 'CaseID', 'IsolationWard'**

3.2 Exploratory Data Analysis

Now, will explore every features, so we can remove unnecessary data-columns and select only those data-columns which has significance to classify the target variable which is **Survived** in our case with **Binary value 0 :Not Survived and 1:Survived** and plot visualizations.

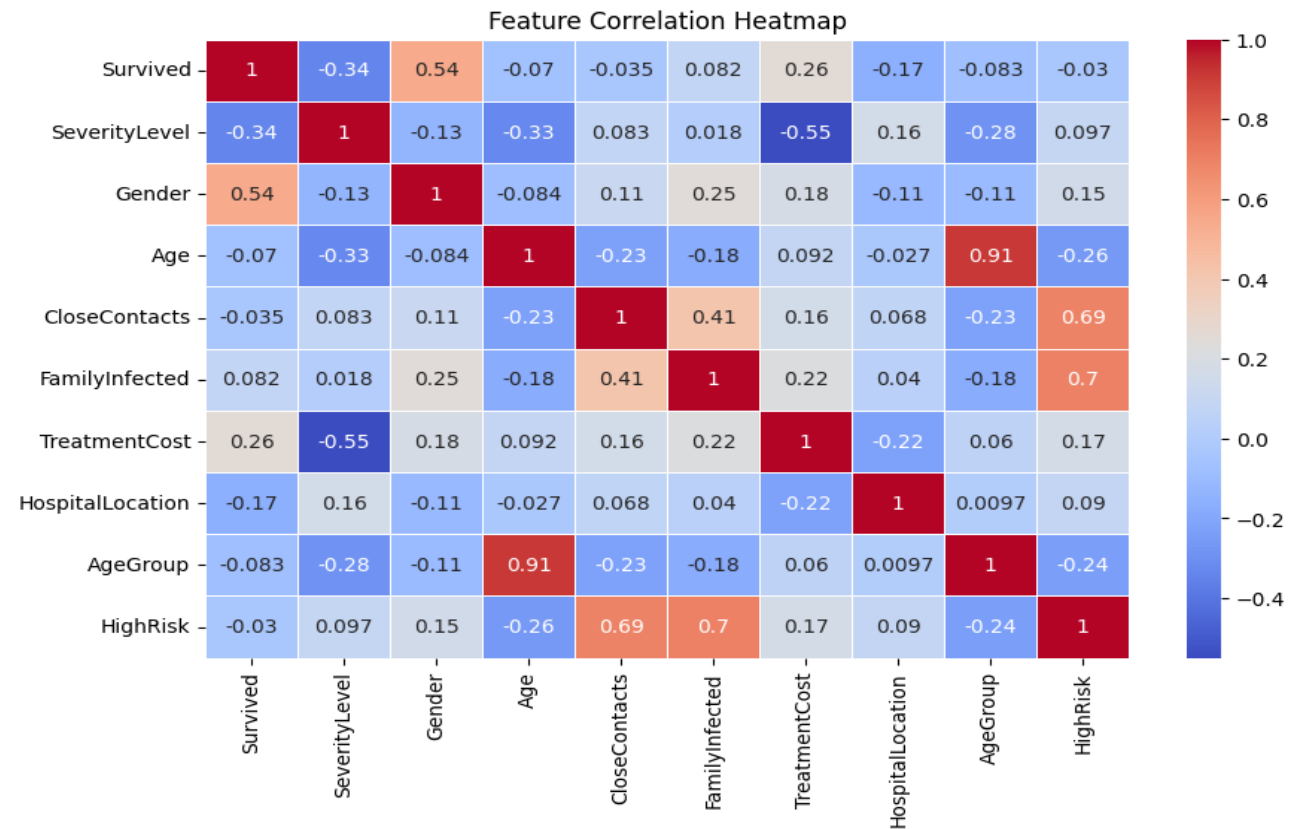


This heatmap visualizes the correlation between different features in the **COVID-19 prediction dataset**. The **color intensity** represents the correlation strength:

● **Positive correlation** (closer to 1)
– Features move in the same direction (e.g., FamilyInfected & CloseContacts).

● **Negative correlation** (closer to -1)
– Features move in opposite directions (e.g., SeverityLevel & TreatmentCost).

○ **Near 0 correlation** – Minimal or no relationship between features (e.g., Survived & HighRisk).
This helps in selecting important features for **machine learning models** and removing redundant ones.



3.3 Feature Engineering

Create new features based on existing ones.

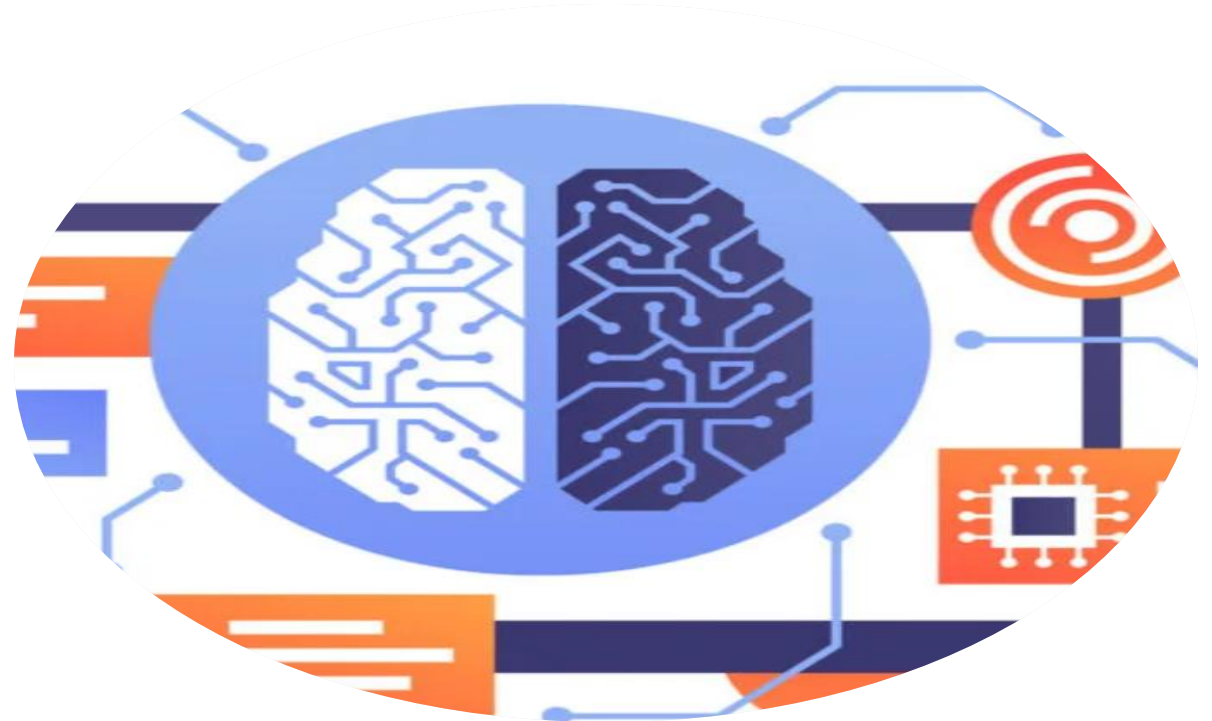
Age Group: Categorize age into groups.

High Risk: Identify patients with more CloseContacts and FamilyInfected.

→	Survived	SeverityLevel	Gender	Age	CloseContacts	FamilyInfected	\
0	0	3	0	22.0	1	0	
1	1	1	1	38.0	1	0	
2	1	3	1	26.0	0	0	
3	1	1	1	35.0	1	0	
4	0	3	0	35.0	0	0	
	TreatmentCost	HospitalLocation	AgeGroup	HighRisk			
0	7.2500		2	1	0		
1	71.2833		0	1	0		
2	7.9250		2	1	0		
3	53.1000		2	1	0		
4	8.0500		2	1	0		

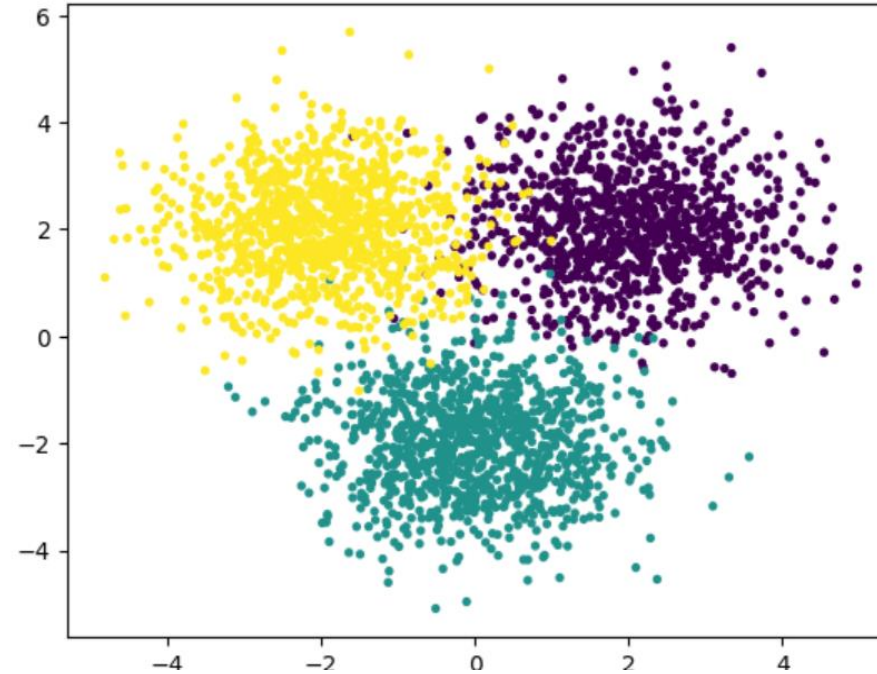
4. ML model

- After conducting the dataset overview and performing data cleaning and preprocessing, the next crucial step is to train an ML model for predictive analysis on unseen data. The dataset is split into 70% for training and 30% for evaluation.
- We employed the KNN classification technique for model prediction, which resulted in an accuracy of 70%.



Why KNN?

- KNN is a simple yet effective supervised machine learning algorithm that works well for both classification and regression tasks. Its primary strength lies in its ability to make predictions based on the similarity between data points.
- KNN is used for Covid-19 prediction because it doesn't assume a specific data distribution, making it flexible for complex medical datasets. It classifies new data based on the similarity to its nearest neighbors. In a Covid-19 prediction model, features like age, symptoms, and medical history are used. The data is split into 70% training and 30% evaluation sets. The model is trained on the training data and tested on the evaluation set, with an accuracy typically around 70%. KNN helps predict the likelihood of infection or the severity of symptoms based on similar cases.



```
✓ [42] accuracy(y_test,y_pred)
```

```
⇒ 0.6977611940298507
```

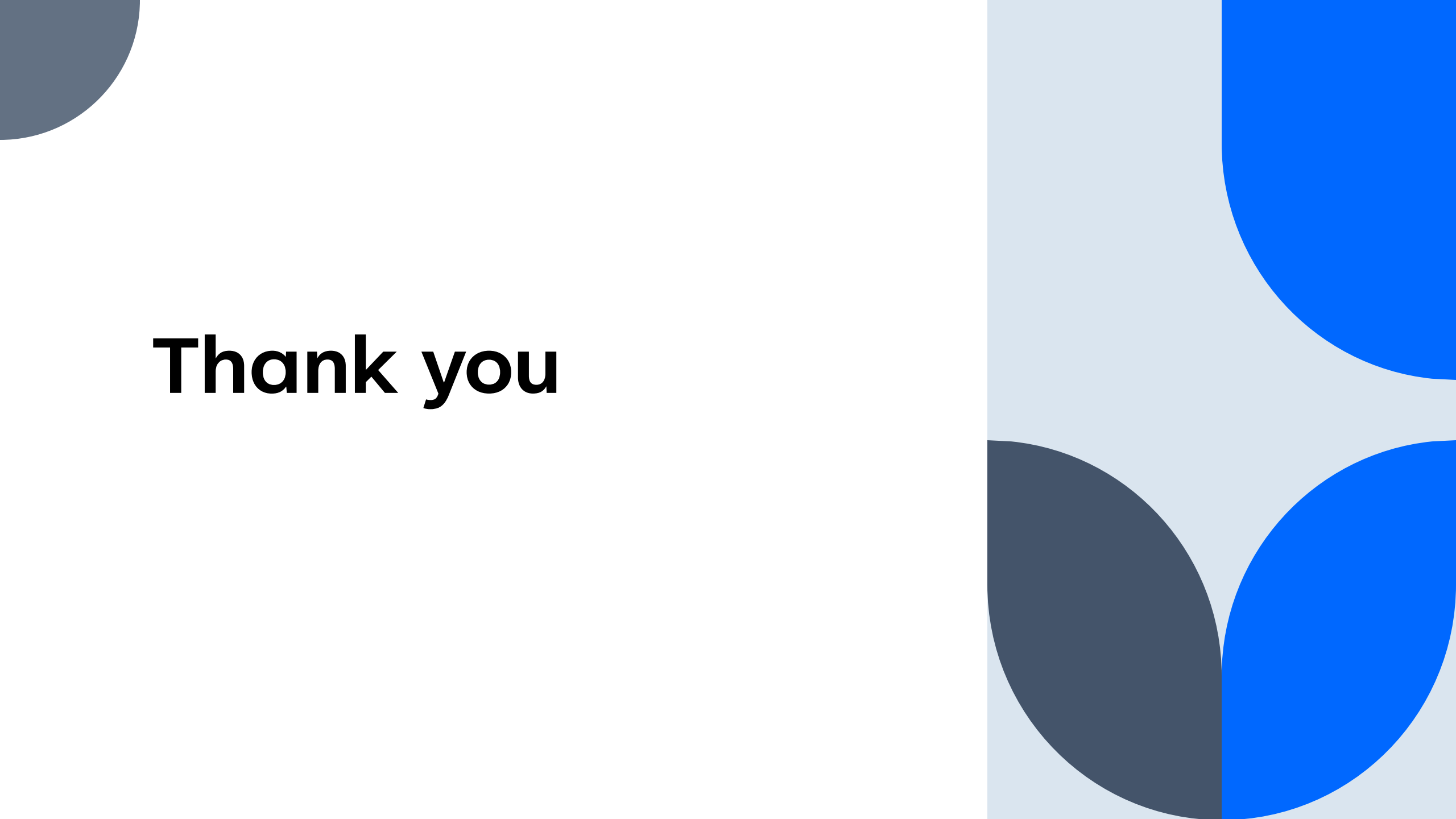
5. Learning and Future Enhancements

We chose KNN for its simplicity and effectiveness in predicting Covid-19 data. However, the accuracy is currently 70%, which is lower than expected. This can be improved in the future by applying the following techniques.

- **Feature Scaling:** Normalize features like age and symptoms to ensure consistency in distance calculations.
- **Optimize K:** Use cross-validation to find the optimal number of neighbors (K) for better performance.
- **Handle Imbalanced Data:** Apply oversampling or undersampling to balance the dataset if there are more non-infected cases.
- **Dimensionality Reduction:** Use PCA to reduce features, focusing on the most important factors, like age and pre-existing conditions.
- **Cross-Validation:** Use k-fold cross-validation to evaluate model performance and avoid overfitting.

These steps can help enhance KNN's performance in predicting Covid-19 outcomes.





Thank you